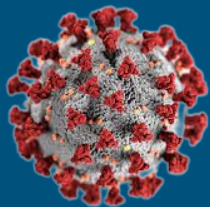


Covid 19 – Classification



A.D. 1308
unipg
UNIVERSITÀ DEGLI STUDI
DI PERUGIA

Tommaso Babbuini [384518]
Matteo Migliorati [382784]
Andrea Jovani [383259]

Descrizione del progetto | Formalizzazione

Di che tipo è il nostro problema?

Nel nostro caso le variabili sono di tipo “Categorical Ordinal”

I dati riportati nel dataset suggeriscono che troviamo un ordine:

None < Mild < Moderate < Severe

Quindi si tratta di una “Ordinal Classification”.

Possiamo usare questa informazione per definire un target.

Ci chiediamo quindi: **Quanto è grave il nostro paziente?**



Descrizione del progetto | Obiettivo



Il nostro obiettivo:

Sviluppare un sistema di intelligenza artificiale in grado di capire la gravità di un paziente basandosi su una lista di sintomi e dati epidemiologici.

Organizzazione delle categorie:

Formalizzeremo i sintomi in macro-categorie (Respiratorie/Sistemiche) per identificare tramite Clustering (K Means) fenotipi clinici naturali, utilizzando poi questi cluster come base affidabile per l'addestramento e la validazione dei modelli di classificazione.



Descrizione del progetto | Analisi dataset

Dataset:

Il file “Cleaned-Data.csv” presenta 27 Feature possiede 316.800 righe. Ogni riga rappresenta un paziente diverso, fatta eccezione per Country il resto dei dati sono binari.

Macro-categorie:

- Sintomatologia
- Dati Demografici
- Contesto Epidemiologico
- Paese di provenienza



Descrizione del progetto | K-means

Sintomi respiratori:

Dry-Cough,
Difficulty-in-Breathing,
Sore-Throat,
Nasal-Congestion,
Runny-Nose

Sintomatologia:

Fever	Tiredness	Dry-Cough	Difficulty-in-Breathing	Sore-Throat	None_Sympton
1/0	1/0	1/0	1/0	1/0	1/0

Pains	Nasal-Congestion	Runny-Nose	Diarrhea	None_Experiencing
1/0	1/0	1/0	1/0	1/0

Dati Demografici:

Age_0-9	Age_10-19	Age_20-24	Age_25-59	Age_60+
1/0	1/0	1/0	1/0	1/0

Gender_Female	Gender_Male	Gender_Transgender
1/0	1/0	1/0

Descrizione del progetto | K-means

Sintomi sistemici/generali:

Fever,
Tiredness,
Pains,
Diarrhea

Sintomatologia:

Fever	Tiredness	Dry-Cough	Difficulty-in-Breathing	Sore-Throat	None_Sympton
1/0	1/0	1/0	1/0	1/0	1/0

Pains	Nasal-Congestion	Runny-Nose	Diarrhea	None_Experiencing
1/0	1/0	1/0	1/0	1/0

Dati Demografici:

Age_0-9	Age_10-19	Age_20-24	Age_25-59	Age_60+
1/0	1/0	1/0	1/0	1/0

Gender_Female	Gender_Male	Gender_Transgender
1/0	1/0	1/0

Descrizione del progetto | K-means

Target originario:

Queste variabili vengono scartate dall'analisi.

Non partecipano al K-Means né alla classificazione, poiché l'obiettivo è generare nuovi target basati esclusivamente sulla sintomatologia reale, ignorando le etichette preesistenti.

Target:

Severity_Mild	Severity_Moderate	Severity_None	Severity_Severe
1/0	1/0	1/0	1/0

Contesto Epidemiologico:

Contact_Dont-Know	Contact_No	Contact_Yes
1/0	1/0	1/0

Provenienza:

Country
Paese di residenza

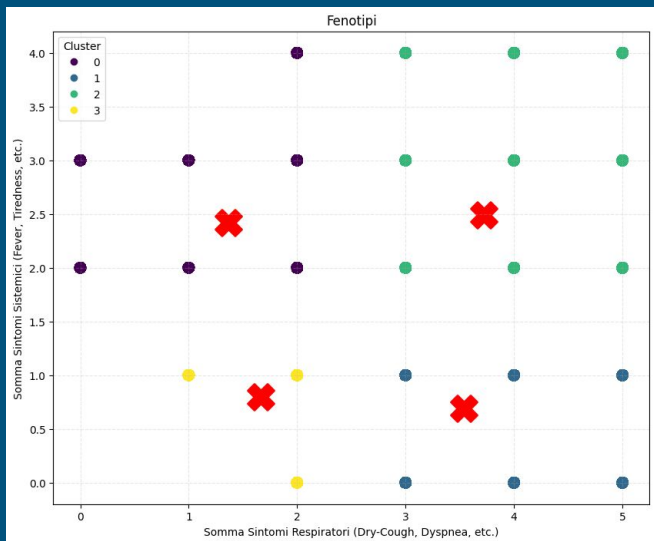
Preparazione dei Dati | Pulizia dati

Pulizia del Dataset:

Selezione delle variabili pertinenti all'analisi clinica ed esclusione dei dati non funzionali **Cosa abbiamo escluso:**

- Country (Provenienza): Rimosso poiché influente sulla dinamica biologica del virus.
- Severity (Target Originale): Escluso in questa fase per permettere un'analisi basata puramente sulla sintomatologia oggettiva.

Preparazione dei Dati | K-means



Come può aiutarci k-means:

Creiamo dei centroidi e **raggruppiamo** i pazienti in cluster basati sulla loro somiglianza, valutiamo i **due gruppi di categorie: respiratori e sistemici/generali**.

Infine alleniamo i classificatori allineandoli sulle categorie restituite da k-means.

Preparazione dei Dati | K-means

Il codice trasforma i singoli sintomi in due categorie respiratori e sistemici.

Successivamente K-Means identifica e raggruppa automaticamente i pazienti in 4 fenotipi distinti, basandosi sulla similarità della loro condizione clinica.

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Caricamento dataset
df = pd.read_csv('Cleaned-Data.csv')

# Definisco i gruppi di sintomi
respiratory_cols = ['Dry-Cough', 'Difficulty-in-Breathing', 'Sore-Throat', 'Nasal-Congestion', 'Runny-Nose']
systemic_cols = ['Fever', 'Tiredness', 'Pains', 'Diarrhea']

# Creo le due nuove feature sommando i sintomi
df['Total_Respiratory'] = df[respiratory_cols].sum(axis=1) # Cambiamo l'asse
df['Total_Systemic'] = df[systemic_cols].sum(axis=1)

# Filtra: tengo solo i pazienti che hanno effettivamente dei sintomi
condition = (
    (df['None_Symptom'] == 0) &
    (df['None_Experiencing'] == 0) &
    ((df['Total_Respiratory'] > 0) | (df['Total_Systemic'] > 0))
)
df_clean = df[condition].copy()
# (Rimuovo chi ha 'None_Symptom' a 1 o somma sintomi a 0)

# Controllo veloce per vedere quante righe ho perso
print(f'Shape originale: {df.shape}')
print(f'Shape dopo filtro: {df_clean.shape}')

# Preparazione dati per KMeans
X = df_clean[['Total_Respiratory', 'Total_Systemic']]

# Standardizzo i dati (necessario per le distanze euclidee del KMeans)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Clustering su 4 gruppi
kmeans = KMeans(n_clusters=4, random_state=775, n_init=300)
df_clean['Phenotype_Cluster'] = kmeans.fit_predict(X_scaled)

# Vediamo come sono distribuiti i cluster
print("\nDistribuzione Cluster:")
print(df_clean['Phenotype_Cluster'].value_counts().sort_index())

... Shape originale: (316800, 29)
Shape dopo filtro: (270000, 29)

Distribuzione Cluster:
Phenotype_Cluster
0      68400
1      76200
2      77400
3      54000
Name: count, dtype: int64
```



Preparazione dei Dati | Training-Test

Divisione dati:

Abbiamo deciso che il dataset andava diviso in 70% dati di Training e 30% di Test (stessa suddivisione usata durante le lezioni).

Seed:

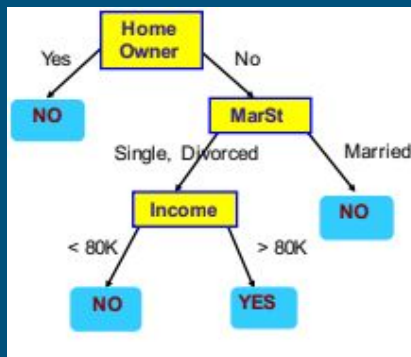
Per il nostro seed impostiamo il valore a 775 (Random_state)
(Serve per garantire un'esecuzione uguale alla nostra).



Classificatori | Decision Tree

Cos'è?

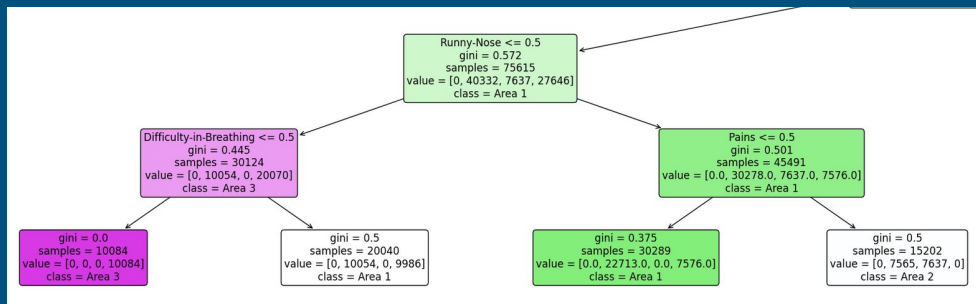
Il Decision Tree è un classificatore che opera secondo una logica gerarchica di tipo "White Box" (Scatola Bianca). A differenza di algoritmi complessi come le reti neurali (Black Box), l'Albero Decisionale rende esplicito ogni passaggio del ragionamento diagnostico.



Classificatori | Decision Tree

Come viene applicato al problema?

L' algoritmo identifica il sintomo più discriminante distinguendo e associando i sintomi alle classi. L' algoritmo parte dal primo sintomo che riscontrano gli infetti e poi assegna un valore True o False in base al peso che assegna l' algoritmo di K-Means.

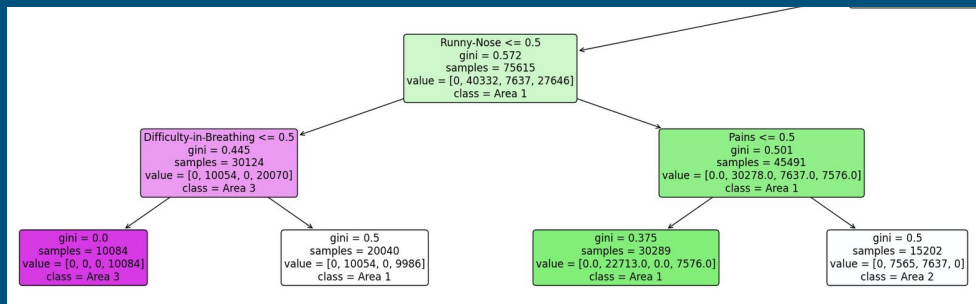


Classificatori | Decision Tree

Come ci aiuta a risolvere il problema?

Abbiamo delle classi binarie (che hanno valore 0 o 1).

Avere due risposte è ottimale per questa tipologia di classificatori che potrebbe risolvere al meglio questa tipologia di problemi.



Classificatori | Decision Tree

Codice Python:

1. Preparazione dei dati
2. Split (train/test)
3. Addestramento classificatore
4. Generazione della Matrice di confusione
5. Visualizzazione della Matrice

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.tree import DecisionTreeClassifier

# X = Sintomi binari (Features)
# y = Cluster del K-Means (Target)
features = [
    'Fever', 'Tiredness', 'Dry-Cough', 'Difficulty-in-Breathing',
    'Sore-Throat', 'Pains', 'Nasal-Congestion', 'Runny-Nose', 'Diarrhea'
]
X = df_clean[features]
y = df_clean['Phenotype_Cluster']

# Dividiamo i dati per testare l'albero su pazienti "mai visti"
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=775)

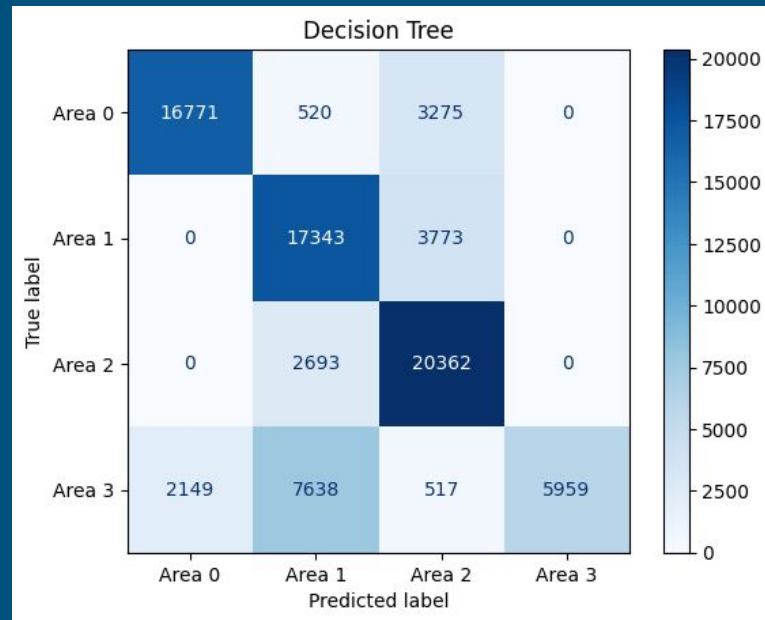
# Addestramento
clf = DecisionTreeClassifier(max_depth=4, random_state=100)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

# Matrice di confusione
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[f'Area {i}' for i in range(4)])
plt.figure(figsize=(10, 8))
disp.plot(cmap='Blues', values_format='d')
plt.title('Matrice di Confusione: Sintomi vs Aree Fenotipiche')
plt.grid(False)
plt.show()
```

Classificatori | Decision Tree

Matrice di confusione:

Questa è la matrice di confusione risultante per il nostro Albero di Decisione con Depth = 4 e Random State = 100, che rappresenta come il modello ha performato.



Classificatori | Random forest

Funzionamento in breve:

Il Random Forest è un algoritmo basato sulla combinazione di numerosi Alberi Decisionali. Mentre il singolo albero può essere impreciso o soggetto a errori, la foresta opera secondo il principio della "saggezza della folla": aggrega i risultati di centinaia di alberi diversi per ottenere una previsione finale più robusta e stabile.



Classificatori | Random forest

Come viene applicato al problema:

L'algoritmo crea diversi sotto-insiemi di dati e sintomi in modo casuale, addestrando un albero su ogni sotto-insieme. Quando deve classificare un nuovo caso, ogni albero esprime un "voto" (0 o 1). La classe che riceve la maggioranza dei voti diventa il responso finale del modello.

Classificatori | Random forest

Utilità per risolvere il problema:

Abbassa la percentuale di errore risolvendo il problema dell'overfitting (sovra-apprendimento) tipico del singolo albero decisorio; è estremamente efficace nel gestire dataset con molti sintomi (variabili), identificando quali di essi hanno il peso maggiore nella diagnosi finale.



Classificatori | Random forest

Codice Python:

1. Preparazione dei dati
2. Split (train/test)
3. Addestramento classificatore
4. Predizioni fatte
5. Report finale

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Usiamo i sintomi originali come X per vedere come influenzano i cluster
# Escludiamo le colonne che abbiamo creato noi (Total_Respiratory, Total_Systemic, Phenotype_Cluster)
features_originali = respiratory_cols + systemic_cols
X_rf = df_clean[features_originali]
y_rf = df_clean['Phenotype_Cluster']

# Split in training e test set
X_train, X_test, y_train, y_test = train_test_split(X_rf, y_rf, test_size=0.3, random_state=775)

# Allenamento del modello con random_state 775
rf = RandomForestClassifier(n_estimators=6, random_state=775)
rf.fit(X_train, y_train)

# Generazione delle predizioni:
# Ogni riga di X_test attraversa tutti gli n_estimators (alberi) della foresta.
# La classe finale in y_pred è determinata dalla maggioranza dei voti espressi dai singoli alberi
y_pred = rf.predict(X_test)

# Report Finale
print("--- Report di Classificazione RF ---\n")
print(classification_report(y_test, y_pred))
```

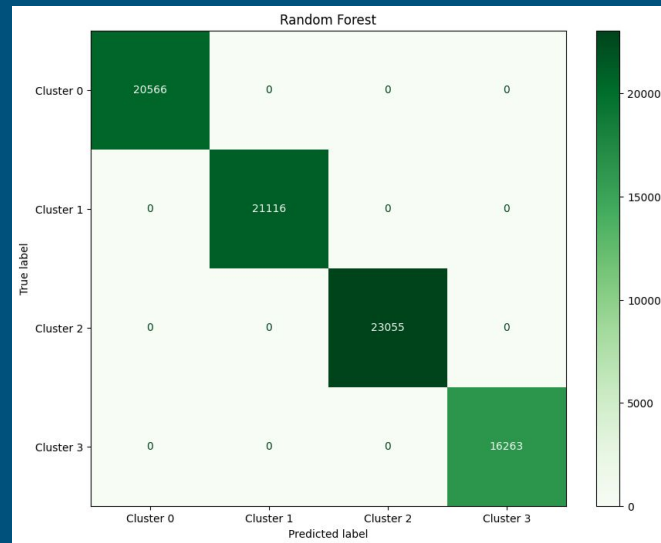
Classificatori | Random forest

Matrice di confusione:

La matrice di confusione mostra una totale corrispondenza tra i cluster previsti e quelli reali.

Il classificatore ha ricostruito perfettamente le regole di assegnazione, senza commettere errori nel test set.

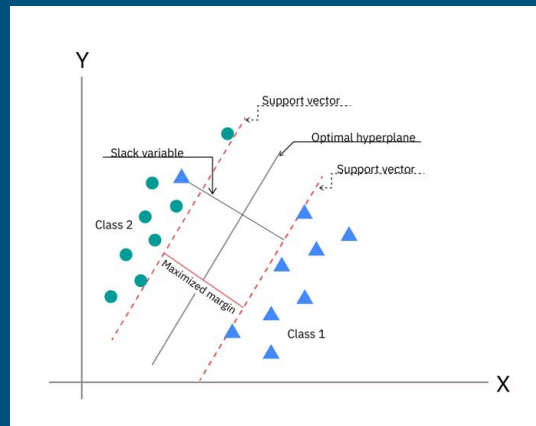
La random forest aveva 6 come $n_estimators$.



Classificatori | Support Vector Machine

Funzionamento?

Operano una distinzione tra due classi trovando l'**iperpiano** ottimale che **massimizza il margine** tra i punti **dati più vicini delle classi opposte**. Il numero di caratteristiche nei dati di input determina se l'iperpiano è una linea in uno spazio **bidimensionale** o un piano in uno spazio **n-dimensionale**.



Classificatori | Support Vector Machine

Come viene applicato al problema:

L'algoritmo proietta i sintomi in uno spazio multidimensionale e cerca la "linea di demarcazione" che separi con maggiore sicurezza i pazienti (es. sani = 0 da quelli malati = 1). Se i dati non sono separabili linearmente, la SVM utilizza una tecnica chiamata Kernel Trick per mappare i dati in dimensioni superiori e trovare comunque una separazione netta.



Classificatori | Support Vector Machine

Utilità per risolvere il problema:

Precisione con margini netti: È imbattibile quando esiste una separazione chiara tra le classi, rendendo la classificazione binaria (0 o 1) estremamente accurata.

Efficienza con **pochi dati**: Funziona molto bene anche **se** il numero di **campioni non è elevatissimo**, purché i sintomi forniti siano significativi per distinguere le **due classi**.



Classificatori | Support Vector Machine

Codice Python:

1. Preparazione dei dati
2. Split (train/test)
3. Scaling
4. Addestramento classificatore
5. Report

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt

# Utilizziamo le feature create in precedenza
features originali = respiratory_cols + systemic_cols
X = df_clean[features originali]
y = df_clean['Phenotype_Cluster']

# Split dei dati
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=775)

# Rendiamo i dati omogenei con lo scaling
scaler_svm = StandardScaler()
X_train_scaled = scaler_svm.fit_transform(X_train)
X_test_scaled = scaler_svm.transform(X_test)

# Usiamo un kernel='linear'
svm_model = SVC(kernel='linear', gamma='scale', random_state=775)
svm_model.fit(X_train_scaled, y_train)

# Report
y_pred_svm = svm_model.predict(X_test_scaled)

print("--- Report di Classificazione SVM ---\n")
print(classification_report(y_test, y_pred_svm))
```

Classificatori | Support Vector Machine

Kernel trick:

Permette di classificare dati che non possono essere separati da una semplice linea retta. Il metodo proietta matematicamente i punti in uno spazio a dimensioni superiori, dove diventa facile dividerli con un piano.

Cerca semplicemente la migliore linea retta (o piano) per separare le classi. È la scelta ideale quando i dati sono già facilmente distinguibili o quando si lavora con dataset molto grandi.

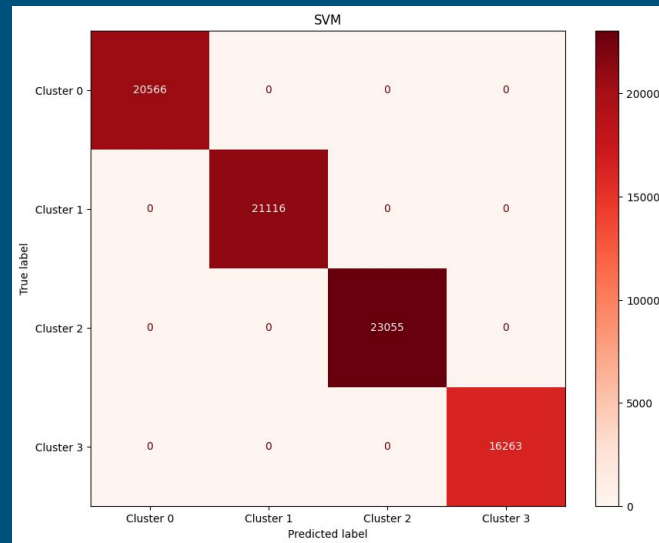


Classificatori | Support Vector Machine

Matrice di confusione:

La matrice di confusione mostra una totale corrispondenza tra i cluster previsti e quelli reali, questo dimostra che classificatore ha ricostruito perfettamente le regole di assegnazione, senza commettere errori nel test set.

La SVM ha un tipo di kernel lineare.



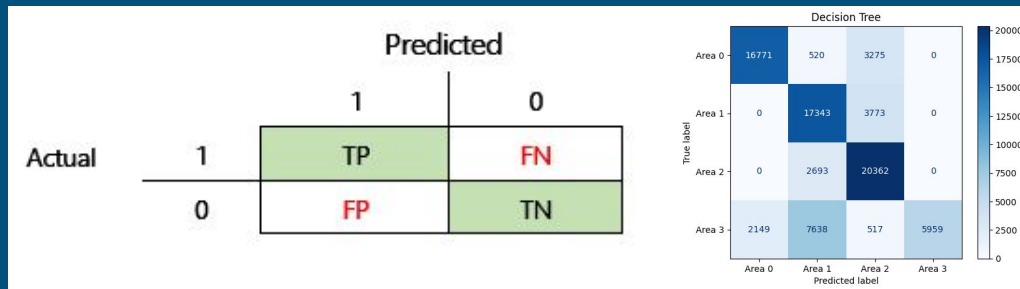
Classificatori | Precision Report

Precision:

Misura l'accuratezza delle predizione positive.

Risponde alla domanda:

“Di tutti gli oggetti che il modello classifica come positivi, quanti sono effettivamente positivi?”



Decision Tree

--- Report di Classificazione DT ---

	precision	recall	f1-score	support
Area 0	0.89	0.82	0.85	20566
Area 1	0.62	0.82	0.70	21116
Area 2	0.73	0.88	0.80	23055
Area 3	1.00	0.37	0.54	16263
accuracy			0.75	81000
macro avg	0.81	0.72	0.72	81000
weighted avg	0.79	0.75	0.73	81000

$$\text{Precision} = \frac{TP}{TP + FP}$$

Report | Dati Decision Tree

Area 0:

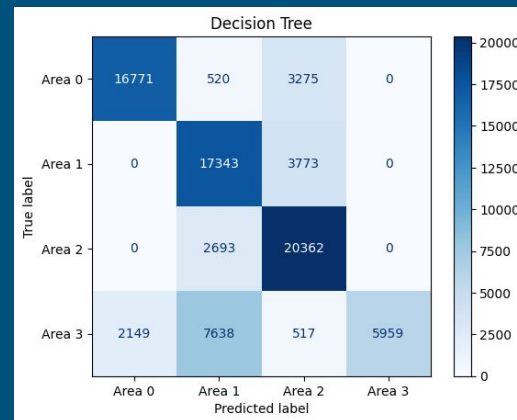
TP (Diagonale) = **16.771**

FP (Colonna 0 - TP)
 $0 + 0 + 2.149 = \mathbf{2.149}$

FN (Riga 0 - TP)
 $520 + 3.275 + 0 = \mathbf{3.795}$

		Predicted	
		1	0
Actual	1	TP	FN
	0	FP	TN

TN (Tutto - TP - FP - FN)
 $(17.343 + 3.773 + 0) +$
 $(2.693 + 20.362 + 0) +$
 $(7.638 + 517 + 5.959) =$
58.285



Report | Precision DT

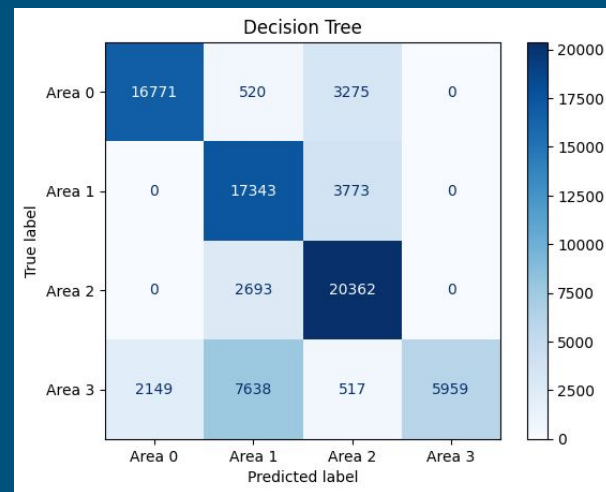
Area 0:

$$\text{Precision} = \frac{TP}{TP + FP}$$

TP = 16.771
FP = 2.149
FN = 3.795

Precision:

$$\begin{aligned} 16.771 / (16.771 + 2.149) &= \\ 16.771 / 18.920 &= \\ \sim 0,89 \end{aligned}$$

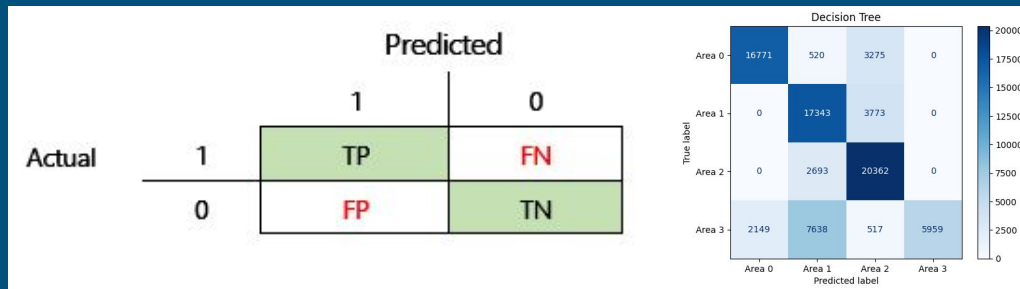


Report | Recall

Recall:

Misura l'abilità del modello nel cercare tutte le istanze positive. Risponde alla domande:

“Di tutte le istanze effettivamente positive, quante sono state identificate correttamente dal modello?”



Decision Tree

--- Report di Classificazione DT ---

	precision	recall	f1-score	support
Area 0	0.89	0.82	0.85	20566
Area 1	0.62	0.82	0.70	21116
Area 2	0.73	0.88	0.80	23055
Area 3	1.00	0.37	0.54	16263
accuracy			0.75	81000
macro avg	0.81	0.72	0.72	81000
weighted avg	0.79	0.75	0.73	81000

$$\text{Recall} = \frac{TP}{TP + FN}$$

Report | Recall DT

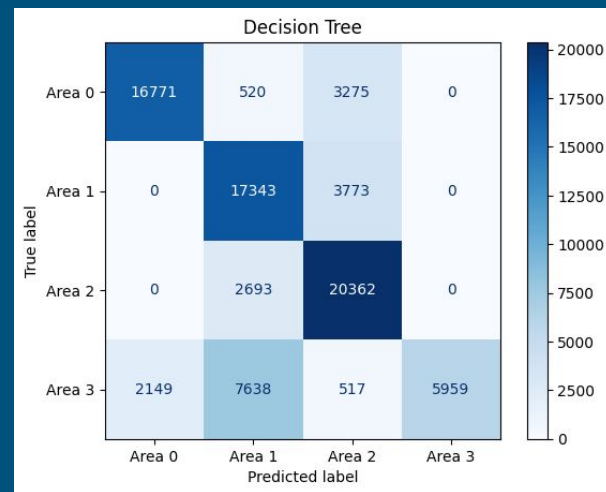
Area 0:

$$\text{Recall} = \frac{TP}{TP + FN}$$

TP = 16.771
FP = 2.149
FN = 3.795

Recall:

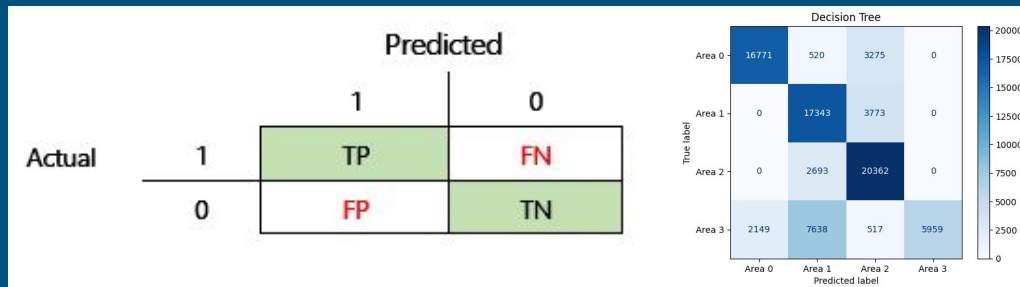
$$\begin{aligned} 16.771 / (16.771 + 3.795) &= \\ 16.771 / 20.566 &= \\ \sim 0,82 \end{aligned}$$



Report | F1-score

F1-score:

La media armonica tra precision e recall. Bilancia le due metriche in un singolo numero, rendendolo particolarmente utile quando precision e recall sono in una situazione di trade-off (compromesso).



Decision Tree

--- Report di Classificazione DT ---

	precision	recall	f1-score	support
Area 0	0.89	0.82	0.85	20566
Area 1	0.62	0.82	0.70	21116
Area 2	0.73	0.88	0.80	23055
Area 3	1.00	0.37	0.54	16263
accuracy			0.75	81000
macro avg	0.81	0.72	0.72	81000
weighted avg	0.79	0.75	0.73	81000

$$\begin{aligned} \text{F1 Score} &= \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} \\ &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$$

Report | F1-score DT

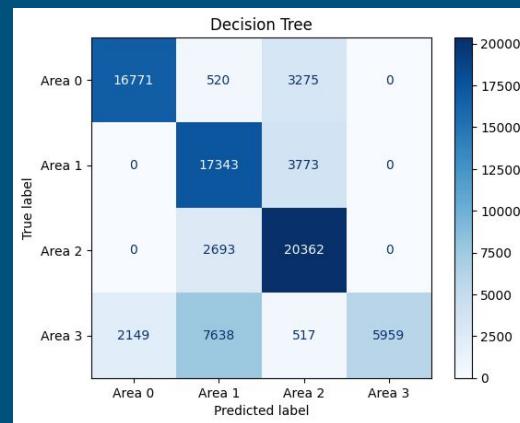
Area 0:

$$\begin{aligned} \text{F1 Score} &= \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} \\ &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$$

F1-score:

$$\begin{aligned} &2 / ((1 / 0.89) + (1 / 0.82)) = \\ &(2 * 0.89 * 0.82) / (0.89 + 0.82) = \\ &\sim 0,85 \end{aligned}$$

Precision = ~0.89
Recall = ~0.82

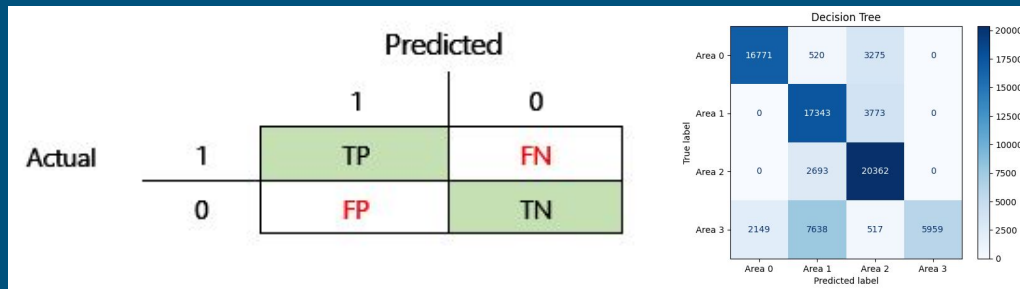


Report | Accuracy

Accuracy:

Misura la correttezza complessiva del modello considerando sia le classi positive che quelle negative.

“Di tutte le predizioni effettuate, quante sono state indovinate correttamente?”



Decision Tree

--- Report di Classificazione DT ---

	precision	recall	f1-score	support
Area 0	0.89	0.82	0.85	20566
Area 1	0.62	0.82	0.70	21116
Area 2	0.73	0.88	0.80	23055
Area 3	1.00	0.37	0.54	16263
accuracy			0.75	81000
macro avg	0.81	0.72	0.72	81000
weighted avg	0.79	0.75	0.73	81000

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Report | Accuracy DT

Area 0:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Nel nostro caso la TN al numeratore sarebbe la somma della diagonale

Accuracy:

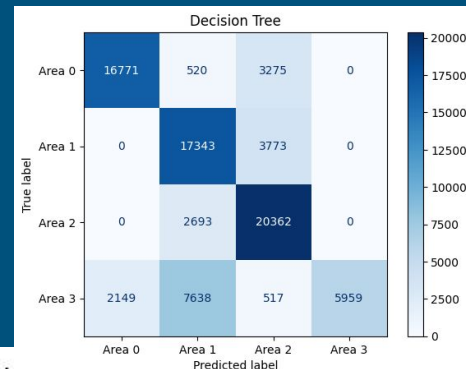
$$\begin{aligned} & (16.771 + 17.343 + \\ & 20.362 + 5.959) / \\ & (16.771 + 58.285 + 2.149 + 3.795) \\ & = 60.435 / 81.000 = \sim 0.75 \end{aligned}$$

TP = 16.771

FP = 2.149

FN = 3.795

TN = 58.285



		Predicted	
		1	0
Actual	1	TP	FN
	0	FP	TN

Confronti | Report

Confronto dei report dei 3 algoritmi:

SVM

--- Report di Classificazione SVM ---

	precision	recall	f1-score	support
0	1.00	1.00	1.00	20602
1	1.00	1.00	1.00	21104
2	1.00	1.00	1.00	23310
3	1.00	1.00	1.00	15984
accuracy			1.00	81000
macro avg	1.00	1.00	1.00	81000
weighted avg	1.00	1.00	1.00	81000

Random Forest

--- Report di Classificazione RF ---

	precision	recall	f1-score	support
0	1.00	1.00	1.00	20566
1	1.00	1.00	1.00	21116
2	1.00	1.00	1.00	23055
3	1.00	1.00	1.00	16263
accuracy			1.00	81000
macro avg	1.00	1.00	1.00	81000
weighted avg	1.00	1.00	1.00	81000

Decision Tree

--- Report di Classificazione DT ---

	precision	recall	f1-score	support
Area 0	0.89	0.82	0.85	20566
Area 1	0.62	0.82	0.70	21116
Area 2	0.73	0.88	0.80	23055
Area 3	1.00	0.37	0.54	16263
accuracy			0.75	81000
macro avg	0.81	0.72	0.72	81000
weighted avg	0.79	0.75	0.73	81000

Confronti | Report DT

Cosa possiamo dire di questi risultati?

K Means ha creato le classi di output basandosi esclusivamente sulle distanze geometriche delle features.

Il limite del decision tree è che può fare solo tagli perpendicolari agli assi (orizzontali o verticali). Non può tracciare una diagonale, è costretto a creare una serie infinita di piccoli gradini.

--- Report di Classificazione SVM ---				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	20602
1	1.00	1.00	1.00	21104
2	1.00	1.00	1.00	23310
3	1.00	1.00	1.00	15984
accuracy			1.00	81000
macro avg	1.00	1.00	1.00	81000
weighted avg	1.00	1.00	1.00	81000

--- Report di Classificazione RF ---				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	20566
1	1.00	1.00	1.00	21116
2	1.00	1.00	1.00	23055
3	1.00	1.00	1.00	16263
accuracy			1.00	81000
macro avg	1.00	1.00	1.00	81000
weighted avg	1.00	1.00	1.00	81000

--- Report di Classificazione DT ---				
	precision	recall	f1-score	support
Area 0	0.89	0.82	0.85	20566
Area 1	0.62	0.82	0.70	21116
Area 2	0.73	0.88	0.80	23055
Area 3	1.00	0.37	0.54	16263
accuracy			0.75	81000
macro avg	0.81	0.72	0.72	81000
weighted avg	0.79	0.75	0.73	81000

Confronti | Report RF

Come si comportano gli altri classificatori?

La **Random Forest** usa diversi di alberi. Ognuno fa la sua "**gradinata**" imperfetta e diversa dagli altri. Mettendoli insieme e facendo la media, i "**gradini**" si **arrotondano** e l'insieme **riesce ad approssimare curve** e diagonali molto meglio del singolo albero.

--- Report di Classificazione SVM ---				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	20602
1	1.00	1.00	1.00	21104
2	1.00	1.00	1.00	23055
3	1.00	1.00	1.00	15984
accuracy			1.00	81000
macro avg	1.00	1.00	1.00	81000
weighted avg	1.00	1.00	1.00	81000

--- Report di Classificazione RF ---				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	20566
1	1.00	1.00	1.00	21116
2	1.00	1.00	1.00	23055
3	1.00	1.00	1.00	16263
accuracy			1.00	81000
macro avg	1.00	1.00	1.00	81000
weighted avg	1.00	1.00	1.00	81000

--- Report di Classificazione DT ---				
	precision	recall	f1-score	support
Area 0	0.89	0.82	0.85	20566
Area 1	0.62	0.82	0.70	21116
Area 2	0.73	0.88	0.80	23055
Area 3	1.00	0.37	0.54	16263
accuracy			0.75	81000
macro avg	0.81	0.72	0.72	81000
weighted avg	0.79	0.75	0.73	81000

Confronti | Report SVM

Come si comportano gli altri classificatori?

I **confini** generati da **K-Means** sono geometricamente **lineari** (iperpiani che separano i centroidi). **Non deve approssimare** nulla: la struttura matematica del classificatore **coincide esattamente**.

Per questo motivo, l'SVM si rivela il classificatore migliore per questo genere di problemi, essendo **matematicamente compatibile**.

--- Report di Classificazione SVM ---				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	20602
1	1.00	1.00	1.00	21104
2	1.00	1.00	1.00	23310
3	1.00	1.00	1.00	15984
accuracy			1.00	81000
macro avg	1.00	1.00	1.00	81000
weighted avg	1.00	1.00	1.00	81000

--- Report di Classificazione RF ---				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	20566
1	1.00	1.00	1.00	21116
2	1.00	1.00	1.00	23055
3	1.00	1.00	1.00	16263
accuracy			1.00	81000
macro avg	1.00	1.00	1.00	81000
weighted avg	1.00	1.00	1.00	81000

--- Report di Classificazione DT ---				
	precision	recall	f1-score	support
Area 0	0.89	0.82	0.85	20566
Area 1	0.62	0.82	0.70	21116
Area 2	0.73	0.88	0.80	23055
Area 3	1.00	0.37	0.54	16263
accuracy			0.75	81000
macro avg	0.81	0.72	0.72	81000
weighted avg	0.79	0.75	0.73	81000

Confronti | Matrici di Confusione

Cos'è?

La matrice di confusione è uno **strumento** che incrocia le etichette reali con le predizioni del modello per **evidenziare** gli **errori** di classificazione.

Come è stato gestito?

I cluster (target) sono stati generati dall'algoritmo K Means utilizzando le stesse variabili numeriche che sono state poi **fornite in input ai classificatori**.

Cos'è successo e perché la SVM sembra migliore?

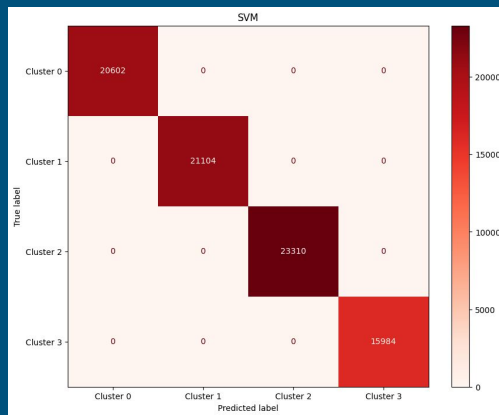
Molto **probabilmente** esiste una **correlazione fortissima** e diretta che il modello ha appreso facilmente.



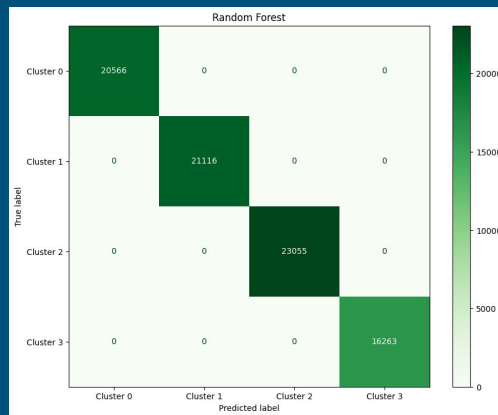
Confronti | Matrici di Confusione

Confronto risultante tra le matrici di confusione generate dai 3 algoritmi:

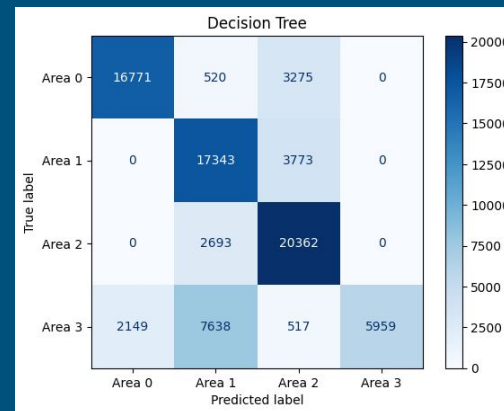
SVM



Random Forest



Decision Tree

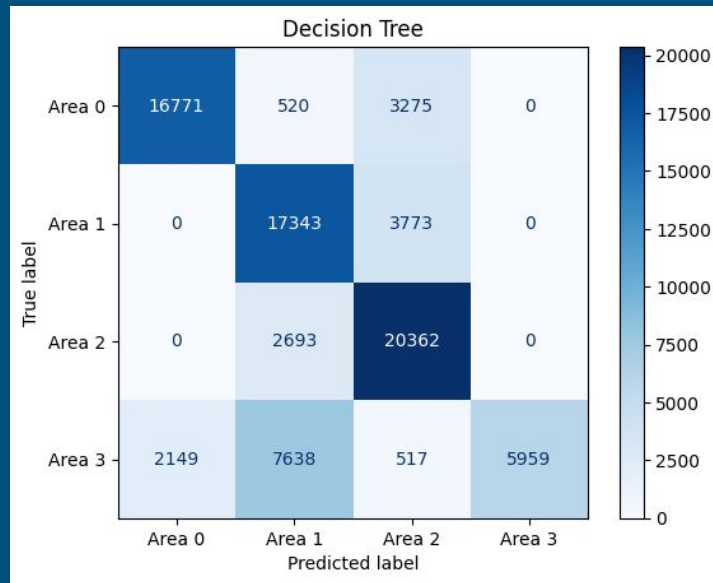


Confronti | Matrici di Confusione DT

Matrici di Confusione DT:

La matrice **non** è perfettamente diagonale e mostra errori (numeri fuori dalla diagonale).

Questi errori non sono casuali, ma concentrati tra cluster vicini: rappresentano i punti situati sul bordo del taglio "a gradini" che l'albero non è riuscito a classificare correttamente a causa dei suoi limiti ortogonali.



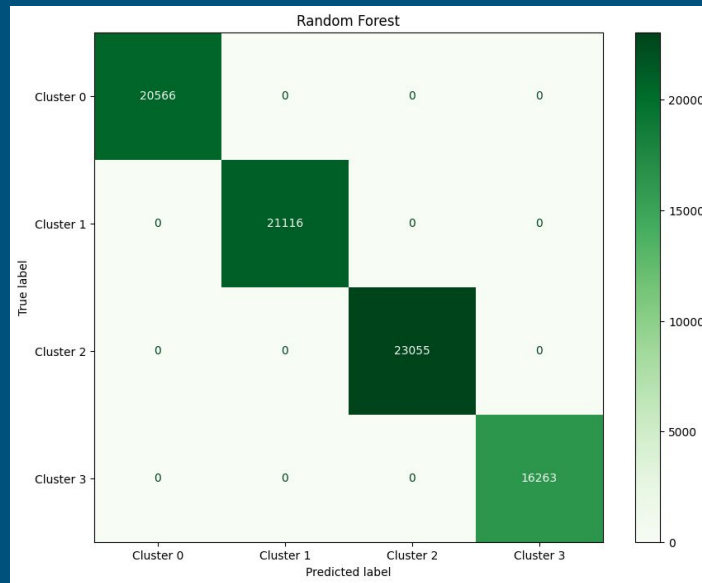
Confronti | Matrici di Confusione RF

Matrici di Confusione RF:

Il **Decision Tree** ha un **limite geometrico** strutturale che viene ereditato dalla Random Forest (Un albero è **incapace** di tracciare bene i confini dei cluster).

Per approssimare questi tagli, l'albero è costretto a generare una **segmentazione** complessa e **frammentata**.

Aggregando alberi la Random Forest riesce a "**smussare**" le spigolosità producendo così un risultato migliore rispetto al Decision Tree.



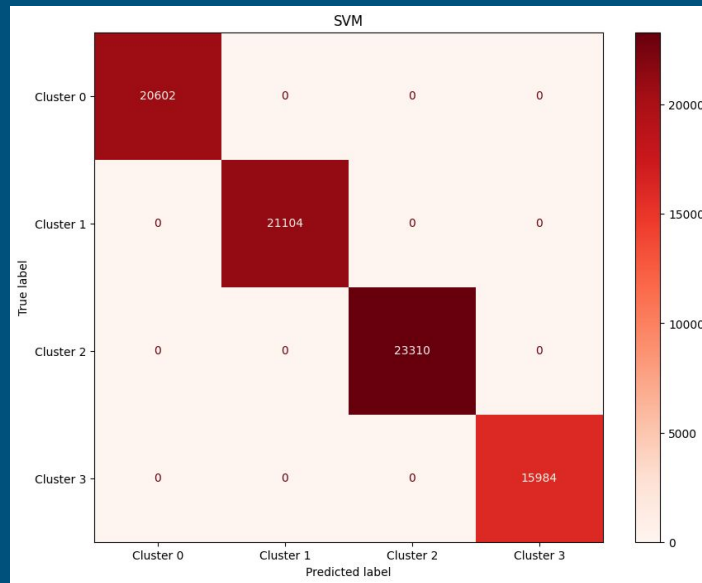
Confronti | Matrici di Confusione SVM

Matrici di Confusione SVM:

Una **SVM** a kernel **lineare** deriva da una perfetta coincidenza strutturale tra il modello predittivo e l'algoritmo usato per produrre i dati.

Avendo utilizzato un classificatore **basato su iperpiani** matematici è riuscito ad **isolare** meglio i cluster.

L'SVM si rivela **ottimale**; operando sulla stessa identica logica geometrica vettoriale **riduce** l'errore drasticamente.





Grazie per l'attenzione!



Tommaso Babbuini
Matteo Migliorati
Andrea Jovani

